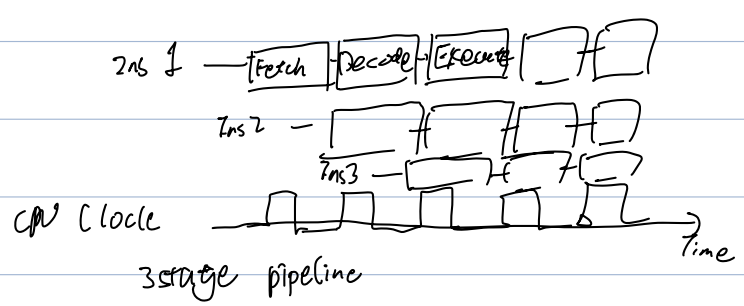
Q1 (a) 32 bit register

(b) $\log_2 8 = 3$ bit address space

(c) $\log_2 4G$ or 32 bit address space

(d) CISC vs RISC

- instruction complexity: CISC's ins is complex and have variable length, so it is quite complex to decode, while the one of RISC's are simple and standardized, so it's simpler for execution.
- code base: Large / small
- Development Time:
- CISC is hardware centric



3 stage pipeline

Fetch: loads an instruction from mem

Decode: identify the instruction to be executed.

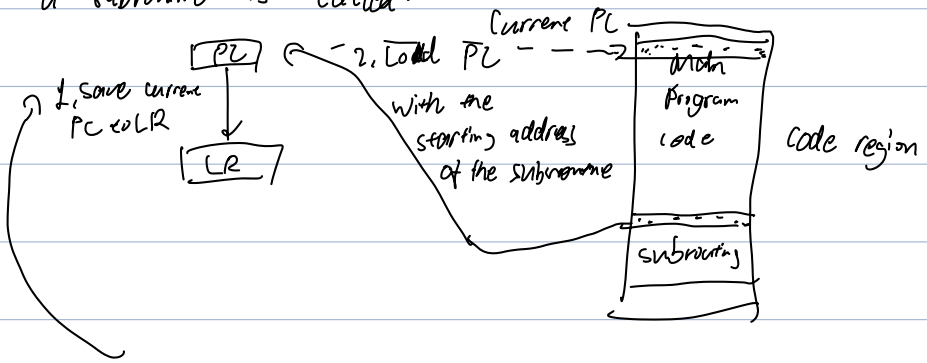
Execute: process the instruction and writes the result back to a register

pipelining improves the performance of CPU by allowing multiple instructions

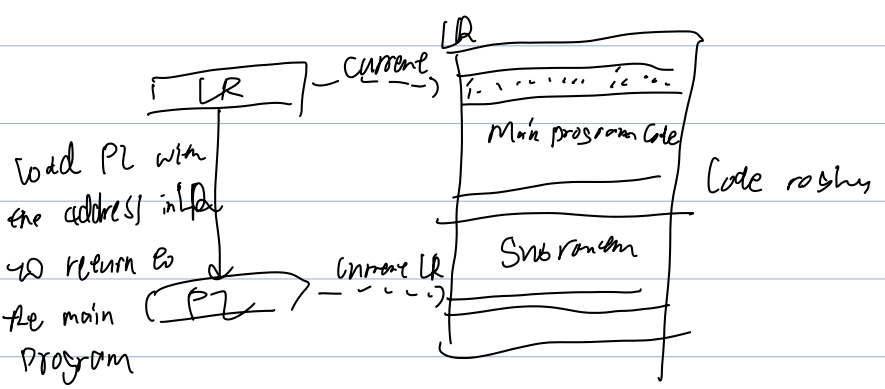
to be executed simultaneously.

SP stack pointer
PC program counter
LR link register

When a subroutine is called:

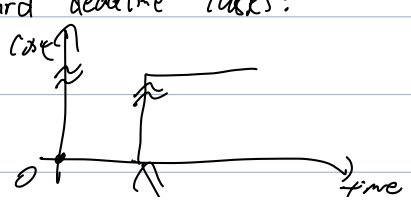


Return



hard vs soft deadline tasks:

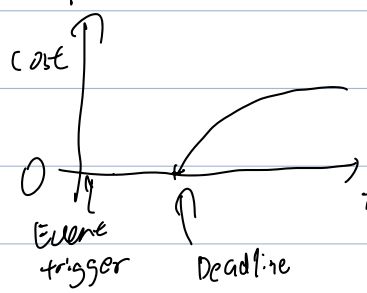
hard deadline tasks:



failure to meee deadline is catastrophic,

The timing coreceness is as important as fonctional coreceness.

Soft deadline tasks:



failure to meee the deadlines is acceptable as long as it is delivered but it will still degrades the performance

Typical example:

hard RT: missile guidance system

Air traffic control system

Critical industrial control system

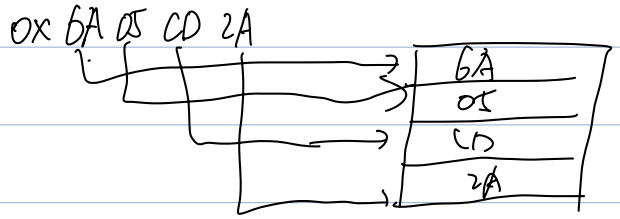
Soft RT: video playing

online gaming

handling input data from keyboard.

Endian: refers to the order of bytes stored in a memory or transmitted through a line.

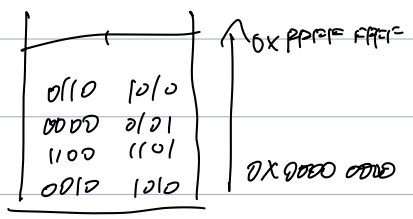
For little endian, LSB is stored in lower address.



MSB LSB

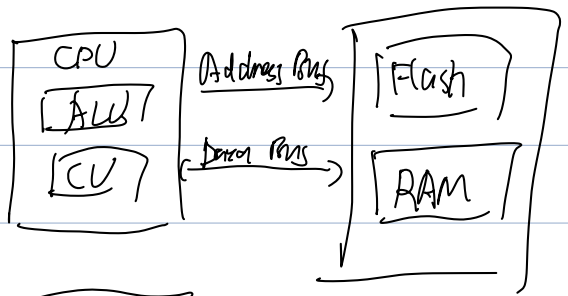
6 A 0 5 C D 2 A

0110 1010 0000 0101 1100 1101 0010 1010

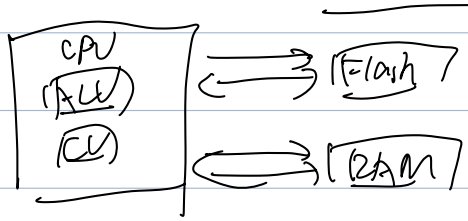


Von Neumann vs Harvard architecture

VN



HV



Von Neumann architecture has one shared program and data memory on the same bus

Harvard architecture has two separate memory and bus for program and data.

	VN	HV
cost	cheap	expensive
CPU	Not able to do same time	can simultaneously
Control circuit	requires less circuit and silicon space	requires more hardware and space.
Memory	Doesn't waste	Can result in waste

Rate monotonic Scheduling RMS

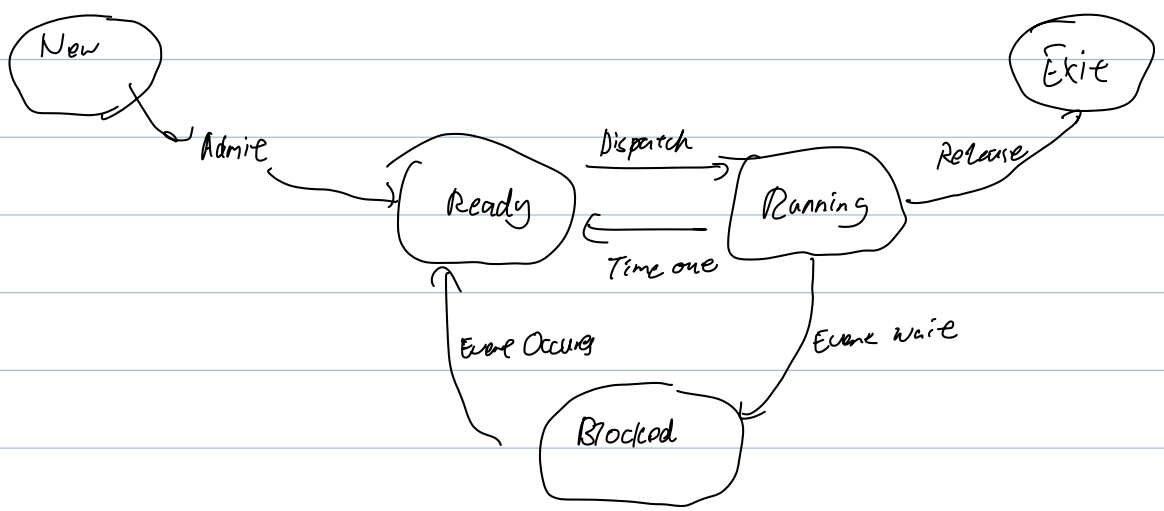
fixed priority preemptive scheduling algorithm commonly used in real-time operating systems for scheduling periodic tasks

For example an autonomous drone

flight stabilization tasks

obstacle detection tasks

telemetry update tasks.



Polling vs interrupt:

in polling, CPU steadily checks the statens of ports at fixed time interval to determine if it needs attention

Interrupt, the device informs the CPU that it requires its attention

Enter

0. button is pressed
1. Finish current instruction (except for lengthy instructions)
2. Push context (8 32bit reg) onto current stack (MSP or PSP)

   context: xPSR, PC, LR, R12, R3, R2, R1, R0
3. Switch to handler mode and use MSP
4. Load PC with address of exception handler
5. Load LR with EXC_RETURN code.
6. Load IPSR with exception number.
7. Start executing code of exception's handler.

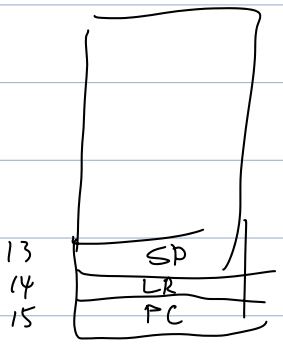
poll vs interrupt.

speed: interrupt is faster, efficient.

↓ don't need to check explicitly steadily

don't waste of CPU time because for the polling the faster & respond we need, the more often we need to check.

Scalability: for the polling, response time depends on all other processing.



Stack pointer stores the current address of stack, which is used to store the content, LR stores the stores the return address of a subroutine or a function call. PC stores the current instruction code. address of the

Least laxity

Task	AT	BT	DDL
1	3	5	12
2	0	9	16
3	6	3	19

	0	1	2	3	4	5	6	7	8	9	10
1	-	-	-	4	4	4	4	4	3	2	-
2	7	7	7	7	6	5	4	3	3	3	2
3	-	-	-	-	-	-	10	9	8	7	6

	11	12	13	14	15	16	17	18	19
1	-	-	-	-	-	-	-	-	-
2	2	2	2	-	-	-	-	-	-
3	5	4	3	2	2	2	-	-	-

Rate monotonic Scheduling

RMS is a fixed priority scheduling algorithm commonly used for real-time OS for scheduling periodic tasks.

每个 Byte 1个地址

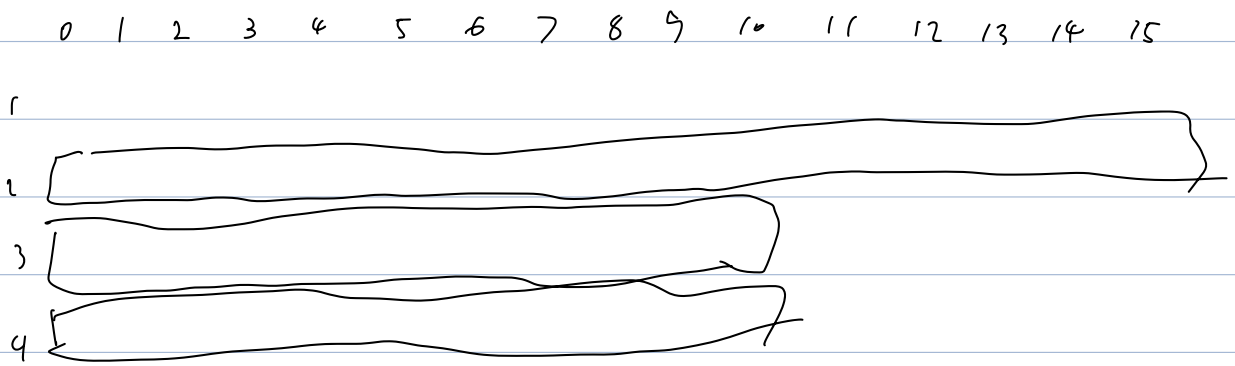
Reg 为 32bit = 4 Byte. 4个地址offset

- 4X 2

+ 4X 4

Low power consumption. low cost, compact size

simple task and less function



Total task utilization is:

$U_T = \sum \frac{C}{T} = \frac{2}{20} + \frac{3}{15} + \frac{2}{10} + \frac{1}{10} = \frac{6+12+12+6}{60} = \frac{6}{10} = 0.6$

Utilization Bound

$U_b = n(2^{\frac{1}{n}} - 1) = 0.757$

$U_T < U_b$ can be scheduled.

$U' = 0.16 + \frac{2}{8} = 0.85 > 0.757$ unknown.

pros: simple to implement

easy to understand.

stable

cons: lower CPU utilization

only deal with independent tasks.

Preemptive: improved responsiveness

predictability

~~efficient CPU Utilization~~

fault tolerance: prevent low-priority tasks from monopolizing CPU